

Tecsup

Diseño y Desarrollo de Software

LABORATORIO N° 08

MVVM - ROOM

DOCENTE:

Juan José León Suiyon

CURSO:

Programación en Móviles
4to Ciclo

LABORATORIO — MVVM + Room

CafeCampus: app de pedidos para la cafetería del campus

Dato	Detalle
Docente	Juan León S.
Tecnología	Kotlin + Jetpack Compose + Material 3 (sin XML), Room 2.6.1, DataStore, Navigation Compose
Patrón	MVVM + Repository (interface)
Duración	1 semana (trabajo guiado en clase + trabajo autónomo)
Entrega	Repositorio GitHub (ramas main y mejora-ia) + README + PROMPTS.md + capturas
Nota	20 puntos (ver rúbrica)

1. El reto

Construirás CafeCampus, una app con dos perfiles en el mismo dispositivo: el Estudiante pide y sigue su pedido, y el Administrador de la cafetería gestiona pedidos y menú. Ambos perfiles comparten la misma base de datos Room, así que cuando el administrador cambia el estado de un pedido, el estudiante lo ve reflejado en su seguimiento.

> **Modo práctica (local)**
> En este laboratorio cada celular tiene su propia base de datos y ambos perfiles conviven en la app. Más adelante el Repository se conectará a un servicio web (Semana 9) y luego a la nube, SIN cambiar ViewModels ni pantallas. Por eso el Repository es una interface. Usa solo datos ficticios (Ley 29733).

Competencias: diseñar entidades y relaciones en Room, escribir DAOs con Flow y suspend, aplicar MVVM con Repository, construir una UI Material 3 fiel a un diseño dado y trabajar con Git de forma profesional.

2. Cómo se trabaja: qué recibes y qué construyes

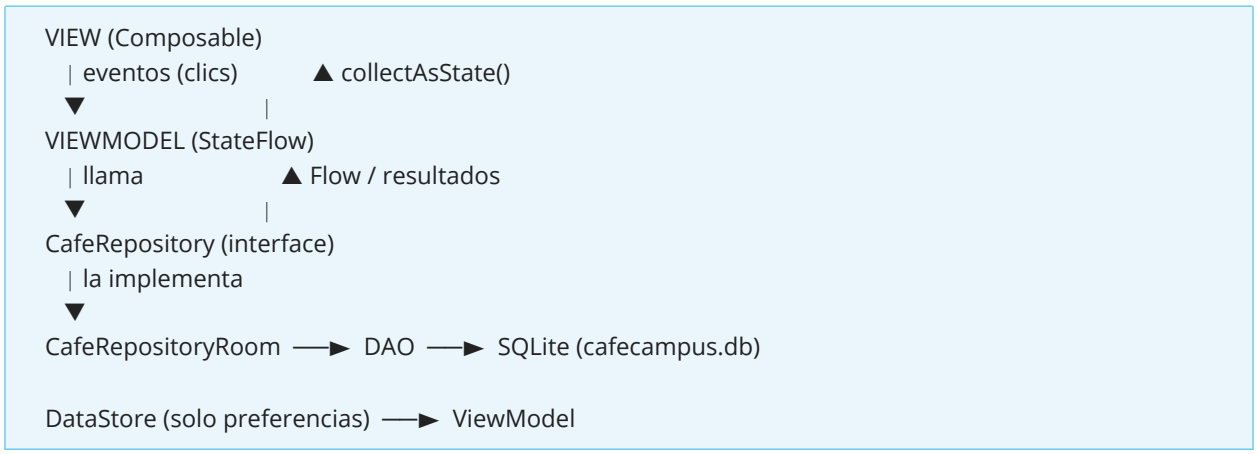
Recibirás un proyecto base que ya compila. Cada archivo del laboratorio tiene un nivel:

Nivel	Significado	Qué haces
ENTREGADO	Ya está hecho en el proyecto base	Léelo, úsalo y no lo modifiques (salvo que el paso lo indique)
GUIADO	Archivo con esqueleto y comentarios // TODO	Completas lo que falta respetando las firmas dadas

Nivel	Significado	Qué haces
TÚ LO HACES	El archivo no existe o está vacío	Lo creas desde cero siguiendo el contrato del paso

> **Ejemplo resuelto**
> El proyecto base incluye UN vertical completo de punta a punta: Producto (entidad), ProductoDao, sus funciones en el Repository, MenuViewModel y MenuScreen. Estúdialo primero: el resto de la app sigue exactamente el mismo patrón.

3. Arquitectura MVVM + Repository



Capa	Conoce a	NO debe
View (ui/)	Solo a su ViewModel	Usar DAO, Repository ni lógica de negocio
ViewModel	Solo al Repository (interface)	Conocer Composables ni el DAO
Repository	Los DAOs (en la implementación)	Exponer entidades de UI ni usar Context de pantalla
DAO / Room	SQL y entidades	Conocer ViewModels

Regla de oro: lectura que debe actualizar la pantalla → Flow (sin suspend). Escritura o acción única → suspend. No se reemplazan, se complementan.

4. Dependencias (ya incluidas en el proyecto base)

Revísalas en build.gradle.kts del módulo app. La versión de KSP debe coincidir con la de Kotlin; si Gradle marca error al sincronizar, revisa primero esto.

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("com.google.devtools.ksp")        // versión = Kotlin (ej. 2.0.21-1.0.28)
```

```
}
dependencies {
    implementation(platform("androidx.compose:compose-bom:2024.09.00"))
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended")
    implementation("androidx.activity:activity-compose:1.9.2")
    implementation("androidx.navigation:navigation-compose:2.8.2")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
    implementation("androidx.datastore:datastore-preferences:1.1.1")
}
```

5. Base de datos: 4 tablas

Tabla	Columnas (tipo)	Relación
usuarios	id (Int, PK auto) · nombre (String) · correo (String) · clave (String) · rol (String: CLIENTE | ADMIN)	—
productos	id (PK auto) · nombre · descripcion · categoria · precio (Double) · disponible (Boolean) · icono (String: cafe, plato, jugo, pan, botella, dulce)	—
pedidos	id (PK auto) · clienteId (FK) · estado (String: RECIBIDO | PREPARANDO | LISTO | ENTREGADO | CANCELADO) · horaRecojo (String) · pago (String: CAJA | YAPE) · nota (String) · total (Double) · fecha (Long, millis)	N pedidos → 1 usuario
detalle_pedido	id (PK auto) · pedidoId (FK) · productoId (Int) · nombreProducto (String) · cantidad (Int) · precioUnitario (Double)	N líneas → 1 pedido

> **¿Por qué guardar nombreProducto y precioUnitario?**
> Es una 'foto' del momento: si mañana el administrador cambia el precio o elimina un producto, los pedidos antiguos no cambian ni se rompen.

Datos iniciales (se cargan solo la primera vez): 6 productos (Café americano S/4.00, Sándwich de pollo S/9.00, Menú del día S/12.00, Jugo de naranja S/5.00, Empanada S/4.50, Chicha morada S/3.50), 1 estudiante (juan@campus.test / 1234) y 1 administrador (admin@cafe.test / admin).

6. Estructura de paquetes y archivos

com.tecsup.cafecampus — marcado con el nivel de cada archivo: [E] entregado, [G] guiado, [A] tú lo haces.

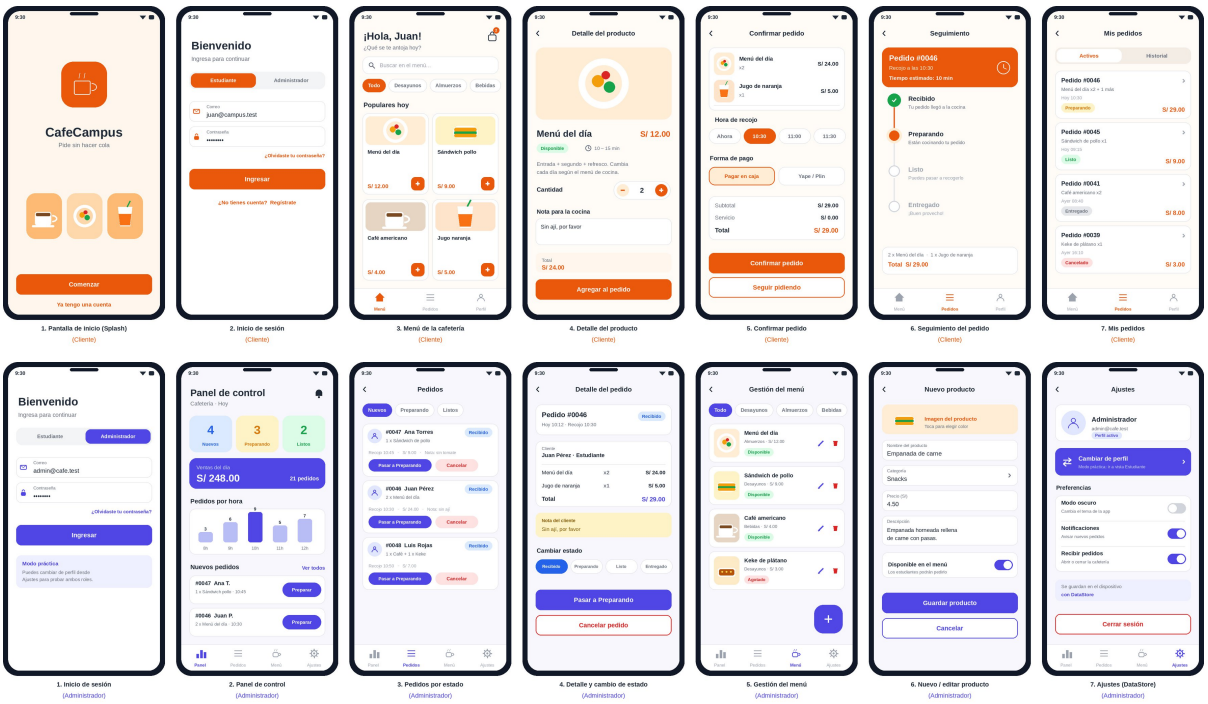
```
com.tecsup.cafecampus
├── MainActivity.kt              [E]
├── CafeApp.kt                   [G]  crea base y repository
```

```
|
├── data
|   ├── local
|   |   ├── entity
|   |   |   ├── Producto.kt          [E]  ← ejemplo resuelto
|   |   |   ├── Usuario.kt           [A]
|   |   |   ├── Pedido.kt            [A]
|   |   |   └── DetallePedido.kt     [A]
|   |   ├── dao
|   |   |   ├── ProductoDao.kt       [E]  ← ejemplo resuelto
|   |   |   ├── UsuarioDao.kt        [A]
|   |   |   └── PedidoDao.kt         [A]
|   |   ├── relation
|   |   |   └── PedidoConDetalle.kt  [A]
|   |   ├── CafeDatabase.kt          [G]
|   |   └── DatosIniciales.kt        [G]
|   ├── repository
|   |   ├── CafeRepository.kt        [E]  interface (el contrato)
|   |   └── CafeRepositoryRoom.kt    [G]  Producto listo; el resto TODO
|   └── prefs
|       └── PreferenciasStore.kt     [A]
|
├── model
|   ├── Rol.kt                   [E]
|   ├── EstadoPedido.kt          [E]
|   └── ItemCarrito.kt           [E]
|
├── viewmodel
|   ├── CafeViewModelFactory.kt      [E]
|   ├── MenuViewModel.kt             [G]  productos listos; carrito TODO
|   ├── SesionViewModel.kt           [A]
|   ├── PedidosClienteViewModel.kt   [A]
|   ├── PanelAdminViewModel.kt       [A]
|   ├── PedidosAdminViewModel.kt     [A]
|   └── MenuAdminViewModel.kt        [A]
|
└── ui
    ├── theme       Color.kt  Type.kt  Theme.kt            [E]
    ├── components  AppTopBar  CafeButton  CafeTextField
    |               EstadoChip  ProductoCard  ProductoIcono
    |               PedidoCard  StepperEstado               [E]
    ├── navigation  Rutas.kt [E]   NavCafe.kt [G]
    ├── cliente
    |   ├── SplashScreen.kt          [E]
    |   ├── MenuScreen.kt            [E]  ← ejemplo resuelto
    |   ├── LoginScreen.kt           [A]
    |   ├── DetalleProductoScreen.kt [A]
    |   ├── ConfirmarPedidoScreen.kt [A]
    |   ├── SeguimientoScreen.kt     [A]
    |   └── MisPedidosScreen.kt      [A]
    └── admin
        ├── PanelScreen.kt          [A]
```

```
├── PedidosAdminScreen.kt       [A]
├── DetallePedidoAdminScreen.kt [A]
├── MenuAdminScreen.kt          [A]
├── FormularioProductoScreen.kt [A]
└── AjustesScreen.kt            [A]
```

7. Diseño: igual al mockup (Material 3)

Tu app debe verse como estas pantallas. Los componentes y el tema ya están entregados: úsalos, no pintes colores sueltos.



1. Pantalla de inicio (Splash) (Cliente)
2. Inicio de sesión (Cliente)
3. Menú de la cafetería (Cliente)
4. Detalle del producto (Cliente)
5. Confirmar pedido (Cliente)
6. Seguimiento del pedido (Cliente)
7. Mis pedidos (Cliente)
1. Inicio de sesión (Administrador)
2. Panel de control (Administrador)
3. Pedidos por estado (Administrador)
4. Detalle y cambio de estado (Administrador)
5. Gestión del menú (Administrador)
6. Nuevo / editar producto (Administrador)
7. Ajustes (DataStore) (Administrador)

Uso	Cliente	Administrador
Color primario	Naranja #EA580C	Índigo #4F46E5
Contenedor	#FFEDD5	#E0E7FF
Fondo	#FFFBF7	#F7F7FD
Estados (comunes)	Recibido azul · Preparando ámbar · Listo verde · Entregado gris · Cancelado rojo	

- CafeTheme(rol) entrega un ColorScheme según el perfil. En tus pantallas usa MaterialTheme.colorScheme.primary, nunca un color fijo.
- Tarjetas con RoundedCornerShape(12.dp) y borde fino, botones de 10 dp, márgenes de 16 dp.
- Scaffold con NavigationBar: 3 ítems en cliente (Menú, Pedidos, Perfil) y 4 en admin (Panel, Pedidos, Menú, Ajustes).
- FilterChip para categorías y estados, LazyColumn/LazyVerticalGrid para listas, FloatingActionButton en la gestión del menú.
- Modo oscuro con darkColorScheme, activable desde Ajustes y guardado en DataStore.

8. El ejemplo resuelto: estúdialo antes de empezar

Producto.kt (entidad = tabla)

```
@Entity(tableName = "productos")
data class Producto(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val descripcion: String,
    val categoria: String,
    val precio: Double,
    val disponible: Boolean = true,
    val icono: String
)
```

ProductoDao.kt (lectura con Flow, escritura con suspend)

```
@Dao
interface ProductoDao {
    @Query("SELECT * FROM productos ORDER BY nombre")
    fun observarTodos(): Flow<List<Producto>>

    @Query("SELECT COUNT(*) FROM productos")
    suspend fun contar(): Int

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(p: Producto): Long

    @Update suspend fun actualizar(p: Producto)
    @Delete suspend fun eliminar(p: Producto)

    @Query("UPDATE productos SET disponible = :d WHERE id = :id")
    suspend fun cambiarDisponibilidad(id: Int, d: Boolean)
}
```

MenuViewModel.kt (Flow → StateFlow con stateIn)

```
class MenuViewModel(private val repo: CafeRepository) : ViewModel() {
    val productos: StateFlow<List<Producto>> = repo.observarProductos()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    // TODO (Paso 7): carrito: agregar, quitar, total, vaciar
}
```

MenuScreen.kt (la View solo observa y avisa)

```
@Composable
fun MenuScreen(vm: MenuViewModel, onProducto: (Int) -> Unit) {
    val productos by vm.productos.collectAsState()
```

```
    LazyVerticalGrid(columns = GridCells.Fixed(2)) {
        items(productos) { p -> ProductoCard(p) { onProducto(p.id) } }
    }
}
```

CafeRepository.kt (el contrato: tú implementas cada función en CafeRepositoryRoom)

```
interface CafeRepository {
    // Usuarios
    suspend fun login(correo: String, clave: String): Usuario?

    // Productos (ejemplo resuelto)
    fun observarProductos(): Flow<List<Producto>>
    suspend fun guardarProducto(p: Producto)
    suspend fun eliminarProducto(p: Producto)
    suspend fun cambiarDisponibilidad(id: Int, disponible: Boolean)

    // Pedidos
    suspend fun crearPedido(clienteId: Int, items: List<ItemCarrito>,
                            horaRecojo: String, pago: String, nota: String): Long
    fun observarPedidosCliente(clienteId: Int): Flow<List<PedidoConDetalle>>
    fun observarPedidosPorEstado(estado: EstadoPedido): Flow<List<PedidoConDetalle>>
    fun observarPedido(id: Int): Flow<PedidoConDetalle?>
    suspend fun cambiarEstado(pedidoId: Int, estado: EstadoPedido)

    // Panel
    fun observarVentasHoy(inicioDia: Long): Flow<Double>
    fun observarCantidadPorEstado(estado: EstadoPedido): Flow<Int>
}
```

FASE 1 · SIN IA · Rama main

Construye CafeCampus paso a paso

Reglas de la Fase 1

No uses IA (ni Gemini ni ChatGPT). Puedes usar la documentación oficial y tus apuntes. Haz un commit por paso, con el mensaje indicado. Si tu commit mezcla varios pasos, pierdes puntos de Git.

Paso 0 · Preparar el repositorio

```
# 1. Descarga el proyecto base y ábrelo en Android Studio
# 2. Sincroniza Gradle y ejecuta: debe abrir el Splash y el Menú
# 3. Crea tu repositorio vacío en GitHub y conéctalo
git init
git branch -M main
git remote add origin https://github.com/TU_USUARIO/CafeCampus.git
git add .
git commit -m "chore: importa proyecto base CafeCampus"
git push -u origin main
```

COMMIT 1 — **chore: importa proyecto base CafeCampus**

La app base debe compilar y mostrar el menú con productos.

Parte A · Datos (Room)

Paso 1 · Entidades Usuario, Pedido y DetallePedido TÚ LO HACES

Objetivo: Crear las 3 entidades que faltan, siguiendo el modelo de Producto.kt y las tablas de la sección 5.

Archivos:

- data/local/entity/Usuario.kt
- data/local/entity/Pedido.kt
- data/local/entity/DetallePedido.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Usuario	@Entity(tableName = "usuarios"); PK autogenerada; rol como String guardando Rol.name
Pedido	@Entity(tableName = "pedidos"); clienteId con ForeignKey hacia Usuario; estado como String (EstadoPedido.name); fecha = System.currentTimeMillis()
DetallePedido	@Entity(tableName = "detalle_pedido"); pedidoId con ForeignKey hacia Pedido con onDelete = CASCADE; guarda nombreProducto y precioUnitario

Elemento	Debe cumplir
Índices	Cada columna FK debe tener su Index para evitar advertencias de Room

> **Pista**
> Una clave foránea se declara en @Entity: foreignKeys = [ForeignKey(entity = Usuario::class, parentColumns = ["id"], childColumns = ["clienteId"])], indices = [Index("clienteId")].

Verifica antes del commit: las entidades compilan (aún no están registradas en la base).

COMMIT 2 **feat(data): agrega entidades Usuario, Pedido y DetallePedido**

Paso 2 · DAOs y relación PedidoConDetalle TÚ LO HACES

Objetivo: Escribir las operaciones de acceso a datos para usuarios y pedidos, y la relación que une un pedido con sus líneas y su cliente.

Archivos:

- data/local/dao/UsuarioDao.kt
- data/local/dao/PedidoDao.kt
- data/local/relation/PedidoConDetalle.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
UsuarioDao	login(correo, clave): Usuario? (suspend) · insertar(u) (suspend) · contar(): Int (suspend)
PedidoDao: escritura	insertarPedido(p): Long · insertarDetalles(lista) · cambiarEstado(id, estado) · todo con suspend
PedidoDao: crear	crearPedido(pedido, detalles): Long marcada @Transaction: inserta el pedido, asigna su id a cada detalle e inserta los detalles
PedidoDao: lectura	observarPorCliente(clienteId), observarPorEstado(estado), observarUno(id) — devuelven Flow de PedidoConDetalle (con @Transaction), ordenados por fecha descendente
PedidoDao: panel	ventasDesde(inicio): Flow<Double> (suma de total sin contar CANCELADO; si no hay pedidos debe dar 0.0) y cantidadPorEstado(estado): Flow<Int>
PedidoConDetalle	@Embedded Pedido + @Relation a la lista de DetallePedido + @Relation al Usuario (cliente)

> **Pista**
> Para sumar sin nulos usa COALESCE(SUM(total), 0.0) en el @Query. Las consultas que devuelven una clase con @Relation necesitan @Transaction.

Verifica antes del commit: el proyecto compila con KSP sin errores en los DAOs.

COMMIT 3 **feat(data): agrega DAOs y relacion PedidoConDetalle**

Paso 3 · CafeDatabase y datos iniciales GUIADO

Objetivo: Registrar las 4 entidades y los 3 DAOs en la base de datos y sembrar los datos de prueba la primera vez.

Archivos:

- data/local/CafeDatabase.kt
- data/local/DatosIniciales.kt
- CafeApp.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Singleton	Una sola instancia de CafeDatabase para toda la app
DatosIniciales	suspend fun sembrar(db): si usuarioDao().contar() == 0 inserta los 6 productos y los 2 usuarios de la sección 5
CafeApp	Application registrada en el AndroidManifest; al iniciar lanza sembrar(db) en una corrutina (Dispatchers.IO)

```
@Database(
    entities = [Producto::class /* TODO: agrega las otras 3 */],
    version = 1,
    exportSchema = false
)
abstract class CafeDatabase : RoomDatabase() {
    abstract fun productoDao(): ProductoDao
    // TODO: usuarioDao() y pedidoDao()

    companion object {
        @Volatile private var INSTANCE: CafeDatabase? = null
        fun obtener(context: Context): CafeDatabase = INSTANCE ?: synchronized(this) {
            // TODO: Room.databaseBuilder(...) con nombre "cafecampus.db" (singleton)
        }
    }
}
```

> **Pista**
> Si cambias una entidad después de ejecutar la app, sube el número de version o desinstala la app del emulador; si no, Room falla porque el esquema no coincide.

Verifica antes del commit: al ejecutar, la app abre sin errores. En Android Studio: View > Tool Windows > App Inspection > Database Inspector, y revisa que existan las 4 tablas con datos.

COMMIT 4 **feat(data): registra entidades y siembra datos iniciales en CafeDatabase**

Paso 4 · CafeRepositoryRoom completo GUIADO

Objetivo: Implementar en CafeRepositoryRoom todas las funciones de la interface (hoy solo las de Producto están hechas; el resto dice TODO).

Archivos:

- data/repository/CafeRepositoryRoom.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
login	Llama al UsuarioDao y devuelve el Usuario o null
crearPedido	Convierte List<ItemCarrito> en DetallePedido (copiando nombre y precio del producto), calcula el total y llama al DAO transaccional
observar*	Solo delegan en los DAOs; no deben usar suspend
cambiarEstado	Guarda estado.name en la tabla
observarVentasHoy	Recibe el inicio del día en millis y delega en ventasDesde

> **Pista**
> Piensa: ¿esta función cambia datos una sola vez o debe avisar cuando cambian? La primera es suspend y la segunda es Flow.

Verifica antes del commit: la clase compila sin ningún TODO() pendiente.

COMMIT 5 **feat(data): implementa CafeRepositoryRoom con todas las operaciones**

Paso 5 · Preferencias con DataStore TÚ LO HACES

Objetivo: Guardar en el dispositivo el perfil activo, el modo oscuro y si la cafetería recibe pedidos.

Archivos:

- data/prefs/PreferenciasStore.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Claves	perfilActivo (String: CLIENTE | ADMIN | vacío), modoOscuro (Boolean), recibirPedidos (Boolean, por defecto true)
Lectura	Un Flow por cada preferencia (perfilActivo, modoOscuro, recibirPedidos)
Escritura	Funciones suspend: guardarPerfil(rol), guardarModoOscuro(b), guardarRecibirPedidos(b), cerrarSesion()
Creación	Extensión val Context.dataStore by preferencesDataStore("ajustes") declarada una sola vez, a nivel de archivo

> **Pista**
> DataStore es para preferencias pequeñas (clave-valor). Los pedidos y productos NO van en DataStore: van en Room.

Verifica antes del commit: compila; la usarás en el Paso 6.

COMMIT 6 **feat(data): agrega PreferenciasStore con DataStore**

Parte B · Sesión y flujo del Cliente

Paso 6 · SesionViewModel y LoginScreen TÚ LO HACES

Objetivo: Permitir el inicio de sesión con selector de rol y recordar el perfil activo.

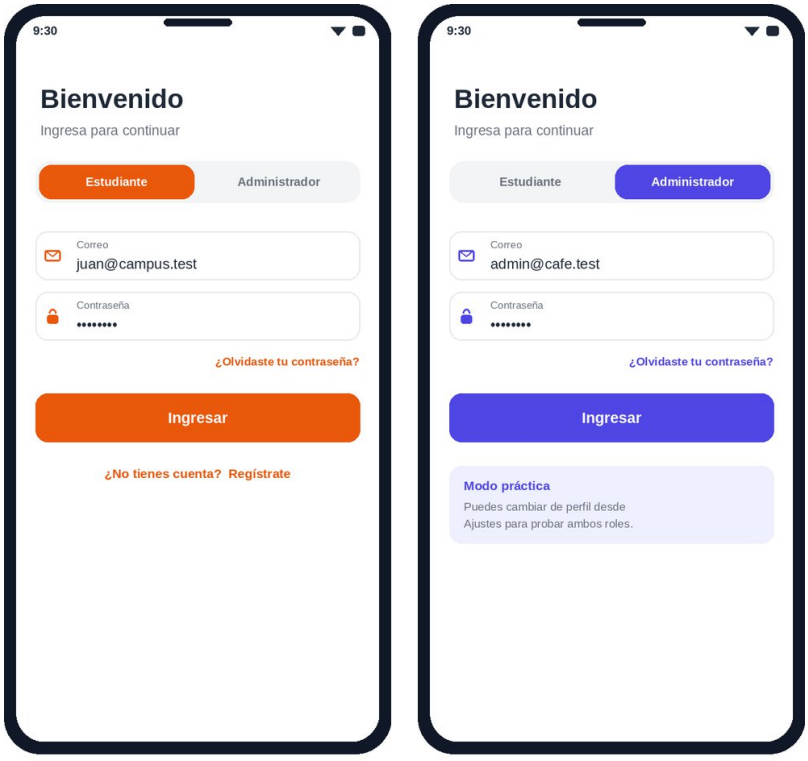
Archivos:

- viewmodel/SesionViewModel.kt
- ui/cliente/LoginScreen.kt
- ui/navigation/NavCafe.kt (agregar el destino)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
SesionViewModel	Expone usuarioActual: StateFlow<Usuario?>, error: StateFlow<String?> y modoOscuro. Función login(correo, clave, rolSeleccionado)
Validaciones	Campos vacíos → mensaje de error. Credenciales incorrectas → 'Correo o contraseña incorrectos'. Si el rol del usuario no coincide con la pestaña elegida → mensaje de error
Éxito	Guarda el perfil en DataStore y navega a Menú (cliente) o Panel (admin)
LoginScreen	Un solo Composable usado por ambos roles: selector Estudiante/Administrador, 2 campos (CafeTextField) y botón 'Ingresar' del color del rol
Splash	Si ya hay perfil guardado, salta el login y va directo a su pantalla principal

Resultado esperado:



2. Inicio de sesión (Cliente)

1. Inicio de sesión (Administrador)

Verifica antes del commit: entras como juan@campus.test / 1234 y como admin@cafe.test / admin; con datos incorrectos aparece el error.

COMMIT 7 **feat(viewmodel): agrega SesionViewModel y login con selector de rol**

Paso 7 · Detalle del producto y carrito GUIADO

Objetivo: Completar el carrito en MenuViewModel y crear el detalle del producto.

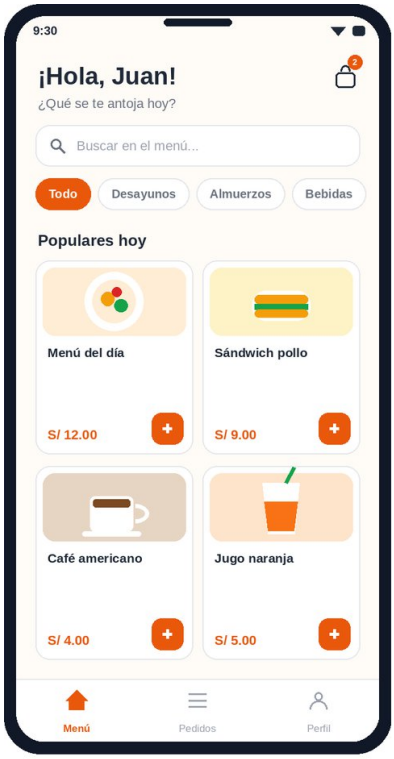
Archivos:

- viewmodel/MenuViewModel.kt (completar los TODO)
- ui/cliente/DetalleProductoScreen.kt

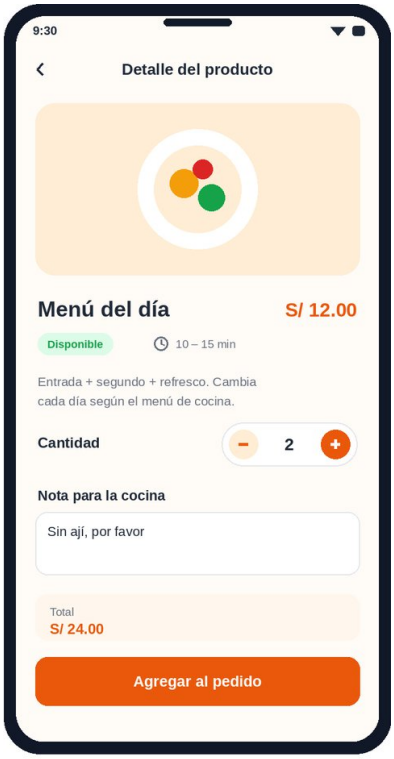
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Carrito	StateFlow<List<ItemCarrito>> con funciones agregar(producto, cantidad, nota), quitar(productoId), vaciar() y total(): Double
Regla	Si el producto ya está en el carrito, suma la cantidad en vez de duplicar la línea
DetalleProductoScreen	Imagen con ProductoIcono, nombre, precio, estado Disponible/Agotado, selector de cantidad (+ / −, mínimo 1), campo de nota y botón 'Agregar al pedido' que muestra el total
Agotado	Si disponible = false, el botón está deshabilitado y se muestra el chip 'Agotado'

Resultado esperado:



3. Menú de la cafetería
(Cliente)



4. Detalle del producto
(Cliente)

Verifica antes del commit: agregas productos desde el detalle y el contador de la bolsa del Menú aumenta.

COMMIT 8 **feat(ui): agrega detalle de producto y carrito en MenuViewModel**

Paso 8 · Confirmar pedido, seguimiento y Mis pedidos TÚ LO HACES

Objetivo: Crear el flujo de pedido del estudiante: confirmar, seguir el estado y ver el historial.

Archivos:

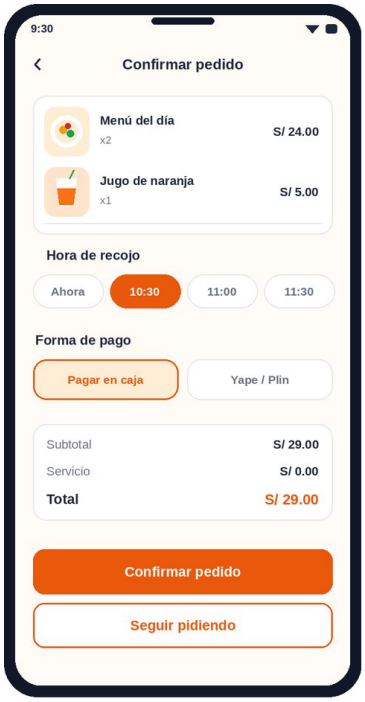
- viewmodel/PedidosClienteViewModel.kt
- ui/cliente/ConfirmarPedidoScreen.kt
- ui/cliente/SeguimientoScreen.kt
- ui/cliente/MisPedidosScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
PedidosClienteViewModel	confirmar(items, horaRecojo, pago, nota) (usa viewModelScope.launch); misPedidos: StateFlow; pedido(id): StateFlow<PedidoConDetalle?>
ConfirmarPedidoScreen	Resumen de líneas, chips de hora de recojo, selección de pago (Caja / Yape-Plin), subtotal y total, botones 'Confirmar pedido' y 'Seguir pidiendo'

Elemento	Debe cumplir
Al confirmar	Crea el pedido con estado RECIBIDO, vacía el carrito y navega al Seguimiento de ese pedido
SeguimientoScreen	Tarjeta de cabecera y StepperEstado (Recibido → Preparando → Listo → Entregado). Se actualiza SOLO cuando el administrador cambia el estado (Flow)
MisPedidosScreen	Pestañas Activos / Historial; cada tarjeta con EstadoChip y total. Activos = RECIBIDO, PREPARANDO, LISTO

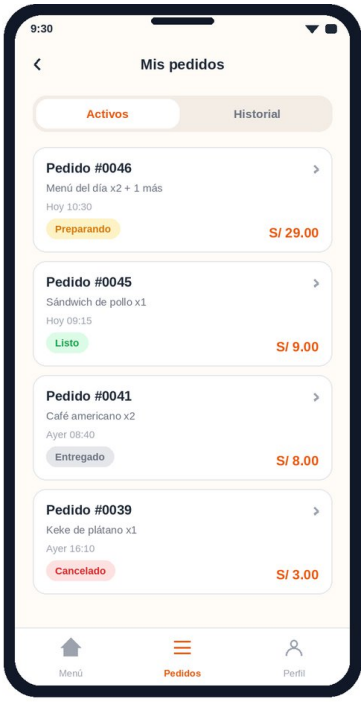
Resultado esperado:



5. Confirmar pedido
(Cliente)



6. Seguimiento del pedido
(Cliente)



7. Mis pedidos
(Cliente)

Verifica antes del commit: haces un pedido con 2 productos y lo ves en Mis pedidos y en Seguimiento con estado Recibido.

COMMIT 9 **feat(ui): agrega confirmar pedido, seguimiento y mis pedidos**

Parte C · Flujo del Administrador

Paso 9 · Panel y pedidos por estado TÚ LO HACES

Objetivo: Crear el panel de control y la pantalla donde la cafetería atiende los pedidos.

Archivos:

- viewmodel/PanelAdminViewModel.kt
- viewmodel/PedidosAdminViewModel.kt
- ui/admin/PanelScreen.kt

- ui/admin/PedidosAdminScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
PanelAdminViewModel	nuevos, preparando, listos: StateFlow<Int> y ventasHoy: StateFlow<Double> (usa el inicio del día actual)
PanelScreen	3 contadores de colores, tarjeta de ventas del día, resumen de 'Nuevos pedidos' con botón Preparar
PedidosAdminViewModel	estadoSeleccionado: MutableStateFlow; pedidos: StateFlow que cambia según el chip (usa flatMapLatest); avanzar(pedido) y cancelar(pedido)
Avanzar estado	RECIBIDO → PREPARANDO → LISTO → ENTREGADO (la lógica vive en EstadoPedido o en el ViewModel, NUNCA en el Composable)
PedidosAdminScreen	FilterChip de estados, tarjetas con cliente, hora de recojo, total, nota y botones 'Pasar a...' y 'Cancelar'

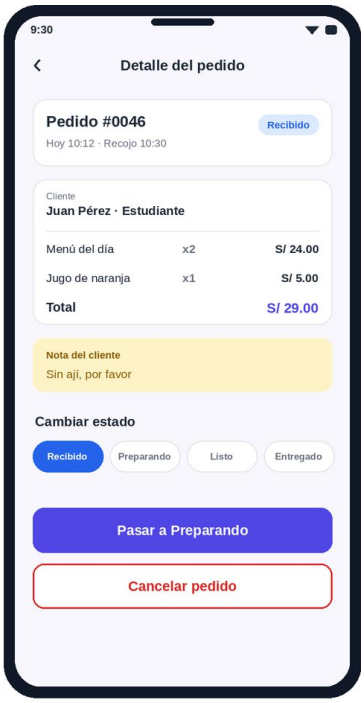
Resultado esperado:



2. Panel de control
(Administrador)



3. Pedidos por estado
(Administrador)



4. Detalle y cambio de estado
(Administrador)

Verifica antes del commit: como cliente haces un pedido; cambias de perfil a admin, lo ves en Nuevos y lo pasas a Preparando.

COMMIT 10 **feat(admin): agrega panel de control y gestion de pedidos por estado**

Paso 10 · Detalle del pedido, gestión del menú y formulario TÚ LO HACES

Objetivo: Permitir al administrador ver el detalle de un pedido y administrar los productos del menú.

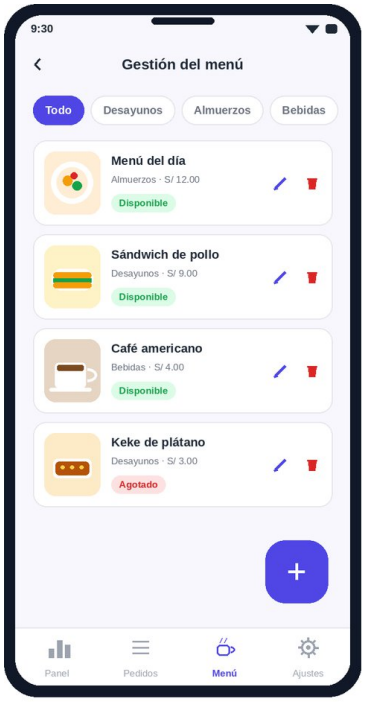
Archivos:

- ui/admin/DetallePedidoAdminScreen.kt
- viewmodel/MenuAdminViewModel.kt
- ui/admin/MenuAdminScreen.kt
- ui/admin/FormularioProductoScreen.kt

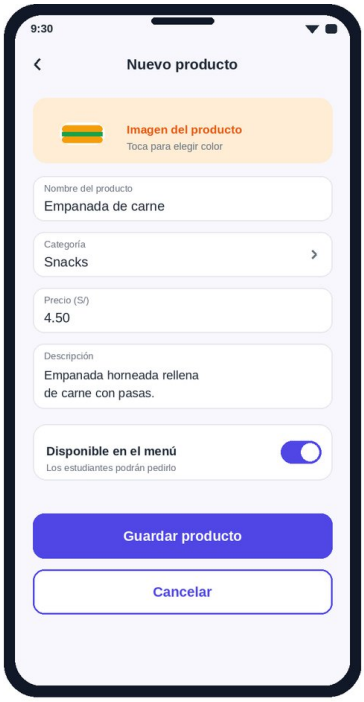
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
DetallePedidoAdminScreen	Cliente, líneas, total, nota del cliente (tarjeta ámbar), selector de estado, botones 'Pasar a...' y 'Cancelar pedido'
MenuAdminViewModel	guardar(producto) (nuevo o editado), eliminar(producto), alternarDisponibilidad(producto)
MenuAdminScreen	Lista con ProductoIcono, categoría, precio, chip Disponible/Agotado, iconos editar y eliminar, FAB para agregar. Eliminar pide confirmación con AlertDialog
FormularioProductoScreen	Campos nombre, categoría, precio, descripción y switch 'Disponible'. Valida: nombre obligatorio y precio mayor que 0. Sirve para crear y para editar (recibe un id opcional)

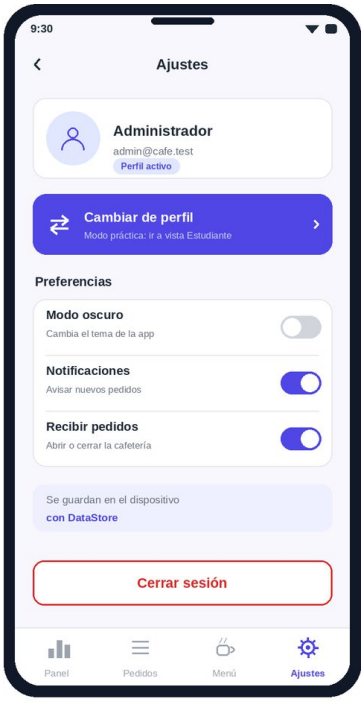
Resultado esperado:



5. Gestión del menú
(Administrador)



6. Nuevo / editar producto
(Administrador)



7. Ajustes (DataStore)
(Administrador)

Verifica antes del commit: marcas un producto como Agotado y el estudiante ya no puede pedirlo; agregas uno nuevo y aparece en el menú del cliente.

COMMIT 11 **feat(admin): agrega detalle de pedido, gestion de menu y formulario**

Paso 11 · Ajustes y cambio de perfil TÚ LO HACES

Objetivo: Cerrar el ciclo: ajustes con DataStore y botón para cambiar de perfil en modo práctica.

Archivos:

- ui/admin/AjustesScreen.kt
- ui/navigation/NavCafe.kt (completar rutas y grafo de cada perfil)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Modo oscuro	El switch guarda en DataStore y cambia el tema de TODA la app al instante (CafeTheme recibe el valor)
Recibir pedidos	Si está apagado, el cliente ve un aviso 'Cafetería cerrada' y no puede confirmar pedidos
Cambiar de perfil	Botón que guarda el otro perfil y navega a su grafo; limpia la pila de navegación (popUpTo)
Cerrar sesión	Borra el perfil y vuelve al Login
Persistencia	Al cerrar y abrir la app, el modo oscuro y el perfil se conservan

Verifica antes del commit: activas modo oscuro, cierras la app por completo y al abrir sigue oscuro; cambias de perfil sin volver a loguearte.

COMMIT 12 **feat(admin): agrega ajustes con DataStore y cambio de perfil**

Paso 12 · Pruebas de aceptación y README

Marca cada prueba en el emulador. Todas deben pasar antes de cerrar la Fase 1.

Prueba	Resultado esperado	OK
Login con clave incorrecta	Mensaje de error, no entra	☑
Estudiante: agregar 2 productos y confirmar	Pedido creado con estado Recibido y total correcto	☑
Producto agotado	No se puede agregar al pedido	☑
Admin: pasar el pedido a Preparando	El seguimiento del estudiante cambia a Preparando sin reiniciar	☑
Admin: cancelar un pedido	Aparece como Cancelado en el historial del estudiante	☑
Admin: cambiar precio de un producto	Pedidos antiguos conservan su precio	☑
Ventas del día	Suma solo pedidos no cancelados	☑
Cerrar y abrir la app	Los pedidos siguen ahí y el perfil se recuerda	☑
Rotar la pantalla	No se pierde el carrito ni los datos	☑

Crea el README.md con: nombre, descripción, árbol de paquetes, el diagrama MVVM, 6 capturas (3 de cada perfil) y las respuestas de la sección 'Para pensar'.

COMMIT 13 **docs: agrega README con estructura y capturas**

Fin de la Fase 1.

```
git add .
git commit -m "docs: agrega README con estructura y capturas"
git push origin main
```

Resumen de commits de la Fase 1

N°	Mensaje del commit	Contenido
1	chore: importa proyecto base CafeCampus	Proyecto base
2	feat(data): agrega entidades Usuario, Pedido y DetallePedido	entity/
3	feat(data): agrega DAOs y relacion PedidoConDetalle	dao/, relation/
4	feat(data): registra entidades y siembra datos iniciales en CafeDatabase	CafeDatabase, DatosIniciales, CafeApp
5	feat(data): implementa CafeRepositoryRoom con todas las operaciones	repository/
6	feat(data): agrega PreferenciasStore con DataStore	prefs/
7	feat(viewmodel): agrega SesionViewModel y login con selector de rol	SesionViewModel, LoginScreen
8	feat(ui): agrega detalle de producto y carrito en MenuViewModel	MenuViewModel, DetalleProductoScreen
9	feat(ui): agrega confirmar pedido, seguimiento y mis pedidos	PedidosClienteViewModel + 3 pantallas
10	feat(admin): agrega panel de control y gestion de pedidos por estado	2 ViewModels + 2 pantallas
11	feat(admin): agrega detalle de pedido, gestion de menu y formulario	MenuAdminViewModel + 3 pantallas
12	feat(admin): agrega ajustes con DataStore y cambio de perfil	AjustesScreen, NavCafe
13	docs: agrega README con estructura y capturas	README.md

FASE 2 · CON IA · Rama mejora-ia

Mejora profesional asistida por Gemini

Cuando la Fase 1 esté completa y subida a main, crea una rama nueva y usa Gemini en Android Studio para mejorar CafeCampus sin romper la arquitectura MVVM + Repository.

```
git checkout -b mejora-ia
```

Mejoras entre las que eliges (mínimo 3)

Mejora	Qué debe lograr	Capas que cambian
Buscar en el menú	Barra de búsqueda que filtra productos por nombre mientras escribes	ViewModel + View
Repetir pedido	Botón en el historial que vuelve a cargar al carrito los productos de un pedido anterior	ViewModel + View
Gráfico real de pedidos por hora	El gráfico del panel se calcula con una consulta Room, no con datos fijos	DAO + Repository + ViewModel + View
Registro de estudiante	Pantalla para crear una cuenta nueva con validaciones (correo único)	DAO + Repository + ViewModel + View
Animaciones y estados vacíos	Transiciones entre pantallas, animación del stepper, mensajes 'No hay pedidos' con ilustración	View
Reporte del día	Pantalla con producto más vendido y total por forma de pago	DAO + Repository + ViewModel + View

Condición obligatoria

Todo lo que genere la IA debe respetar MVVM: la View no usa DAO ni Repository; el ViewModel solo conoce la interface CafeRepository; si agregas una función al Repository, la declaras primero en la interface. Si Gemini rompe esta regla, lo corriges y lo anotas en PROMPTS.md.

Ejemplo de prompt (puedes mejorarlo)

```
Tengo una app Kotlin con Jetpack Compose, Material 3, MVVM, Room y un
CafeRepository (interface). Agrega una barra de busqueda en MenuScreen
que filtre productos por nombre. La logica del filtro debe ir en
MenuViewModel (combine de StateFlow), NO en el Composable. No uses XML.
Muestrame solo los archivos que cambian.
```

Commits mínimos de la Fase 2

Mínimo 3 commits en la rama mejora-ia, uno por mejora, con el prefijo 'ia:'.

COMMIT A	ia: agrega busqueda de productos en el menu

COMMIT B **ia: agrega repetir pedido desde el historial**

COMMIT C **ia: agrega grafico real de pedidos por hora**

COMMIT D **docs: agrega PROMPTS.md**
Documento con todos los prompts usados.

```
git push -u origin mejora-ia
```

Contenido de PROMPTS.md

```
# Prompts utilizados

## Prompt 1
**Texto enviado a Gemini:** ...
**Que genero la IA:** ...
**Que tuve que corregir yo:** ...
**Respeta MVVM?:** Si / No (explica)

## Reflexion (5 lineas)
- Que mejoro la IA y que no?
- Que error cometio?
- Que aprendi sobre MVVM y Room?
```

9. Para pensar

Responde con tus palabras en el README (máximo 6 líneas en total).

Pregunta	Tu respuesta
¿Por qué el Repository es una interface?	
¿Por qué crearPedido es suspend y observarPedidosCliente es Flow?	
¿Qué pasaría si dos perfiles usaran bases de datos distintas?	
¿Qué debería cambiar si mañana los datos viven en un servidor?	

10. Entrega

- Enlace al repositorio GitHub público con las ramas main y mejora-ia.
- main: mínimo 13 commits (Fase 1). mejora-ia: mínimo 3 commits + PROMPTS.md (Fase 2).
- README.md con estructura de paquetes, capturas de ambos perfiles y respuestas de la sección 9.
- Fecha límite: según el cronograma del curso.

Desarrollo y evidencias CafeCampus

Estudiante: Jery Becerra Ninaquispe. Sección A. Se utilizó exclusivamente la primera guía GLAB-S08-JLEONS-2026-1.pdf. Esta sección forma parte del mismo documento y completa el procedimiento, las evidencias y las conclusiones.

Se revisó y verificó el proyecto existente SEMANA8/CafeCampus del repositorio Programaciones_moviles_Becerra. La implementación utiliza Kotlin, Jetpack Compose, Material 3, Room 2.6.1, Repository como interface, ViewModels con StateFlow y preferencias DataStore.

La rama main pasó 5 pruebas instrumentadas, incluida la compra con dos productos, cambio de estado por el administrador, seguimiento del estudiante, conservación del carrito al recrear Activity, modo oscuro y bloqueo de confirmación al cerrar la recepción. Las pruebas de Room verifican también precio histórico, cancelación, ventas sin cancelados y rechazo de agotados.

La rama mejora-ia compiló, pasó 5 pruebas unitarias de PlanRecreo y 6 pruebas instrumentadas. Las mejoras comprobadas incluyen búsqueda sin distinguir tildes, repetición desde historial, gráfico de pedidos por hora y estimación Mi recreo con supuestos explícitos.

Se conservó el historial real de commits de ambas ramas. No se crearon commits vacíos para simular avances. La implementación previa ya declaraba asistencia de ChatGPT en main y en mejora-ia: no se presenta la fase inicial como elaborada sin IA ni se atribuye a Gemini un trabajo que no realizó.

Repositorio base:
https://github.com/JeryBn/Programaciones_moviles_Becerra/tree/main/SEMANA8/CafeCampus

Mejoras: https://github.com/JeryBn/Programaciones_moviles_Becerra/tree/mejora-ia/SEMANA8/CafeCampus

Verificación de los pasos de la guía

Pasos 1 a 3. Existen cuatro entidades y tres DAOs. PedidoConDetalle usa relaciones; las columnas foráneas tienen índices. La creación de la cabecera y sus líneas es transaccional y la siembra inicial es idempotente.

Pasos 4 y 5. CafeRepositoryRoom implementa el contrato de repositorio. Las lecturas observables devuelven Flow y las escrituras son suspend. DataStore guarda perfil, modo oscuro y recepción de pedidos.

Pasos 6 y 7. Login comprueba campos, credenciales y rol. El carrito combina cantidades del mismo producto y calcula el total; un producto agotado no puede confirmarse.

Paso 8. Confirmar crea RECIBIDO, conserva nombres y precios históricos, vacía el carrito y navega al seguimiento. Activos e historial dependen de los estados del pedido.

Pasos 9 y 10. El administrador observa contadores y pedidos por estado, avanza el flujo y gestiona el menú. Room comparte los cambios con el estudiante mediante Flow.

Pasos 11 y 12. El cambio de perfil recrea el grafo de navegación. El modo oscuro se conserva al recrear Activity y la recepción cerrada deshabilita confirmar. Los pedidos confirmados permanecen en Room; el carrito en ViewModel se conserva al cambiar configuración, pero no se afirma que sobreviva a la muerte del proceso.

Respuestas de la sección Para pensar

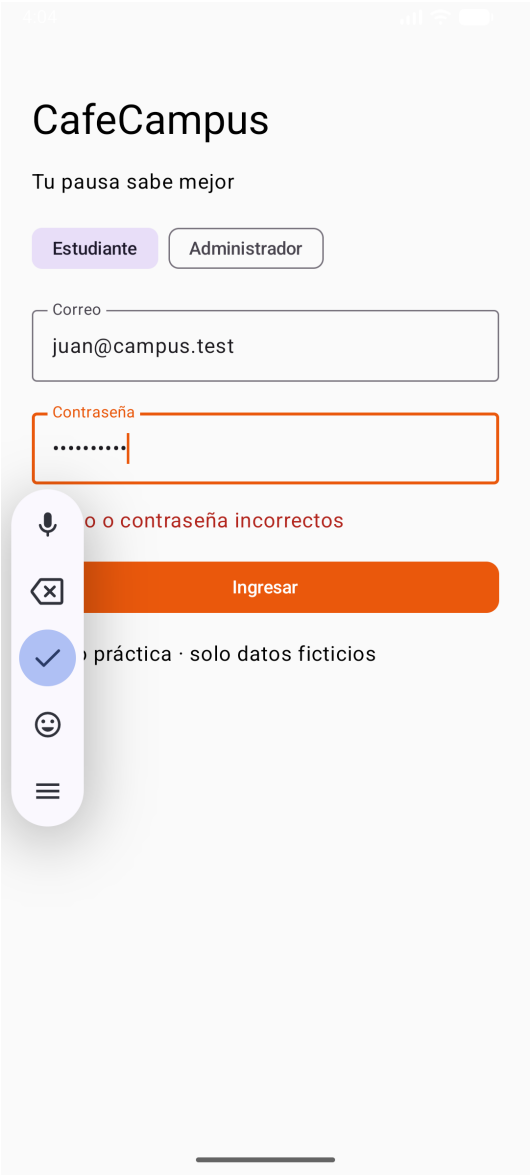
El Repository es una interface para cambiar la fuente de datos sin modificar los ViewModels.

crearPedido es suspend por ser una escritura única; observarPedidosCliente es Flow porque debe emitir cambios.

Con bases distintas el administrador y el estudiante no observarían el mismo pedido actualizado.

Con un servidor cambiaría la implementación del Repository y la sincronización, manteniendo el contrato.

Inicio de sesión y menú del estudiante





Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

Detalle del producto y confirmación

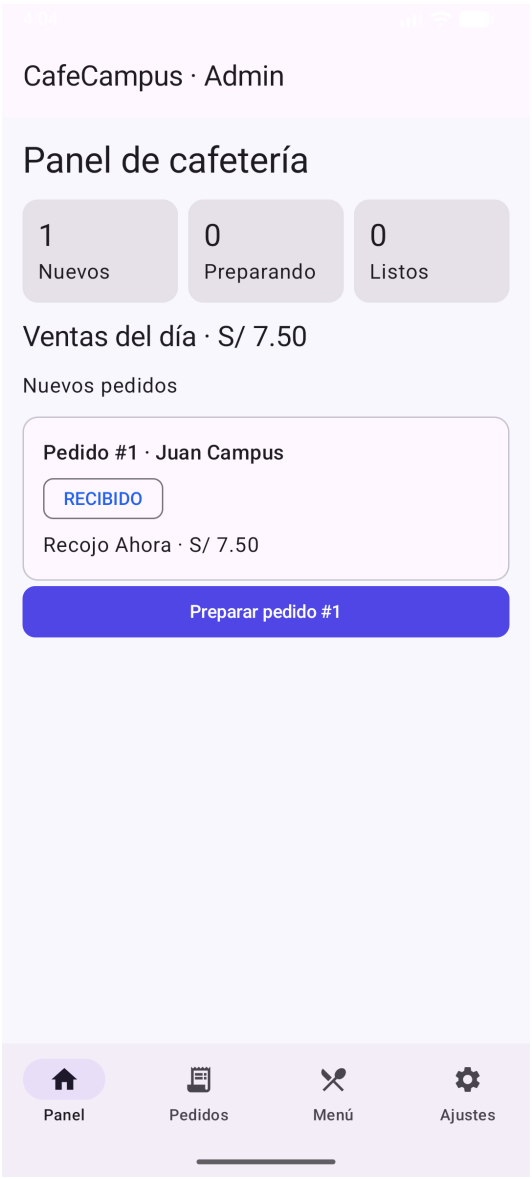




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

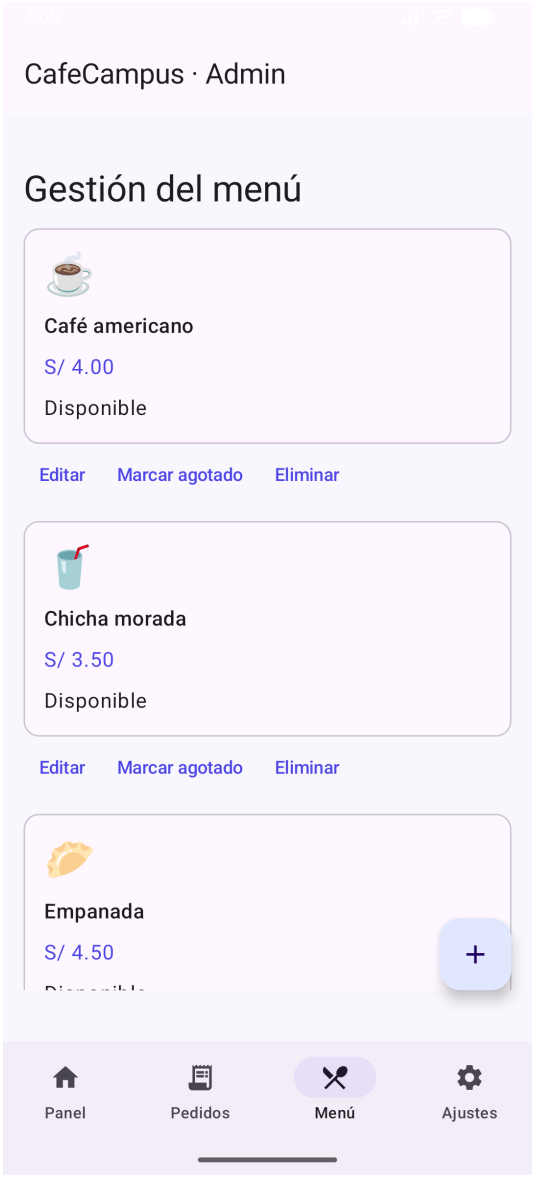
Seguimiento y panel de administración

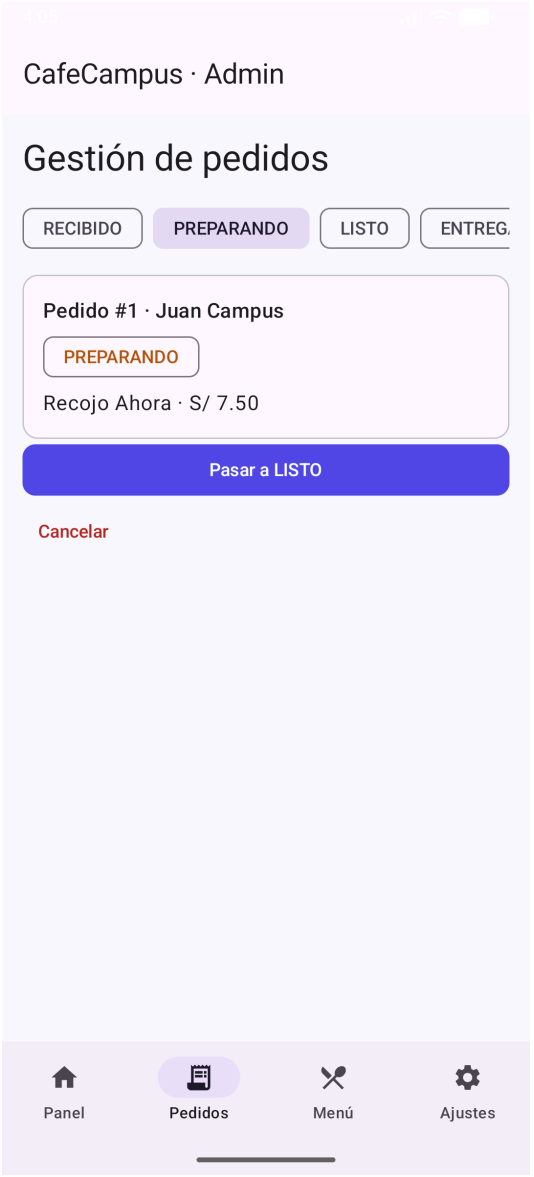




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

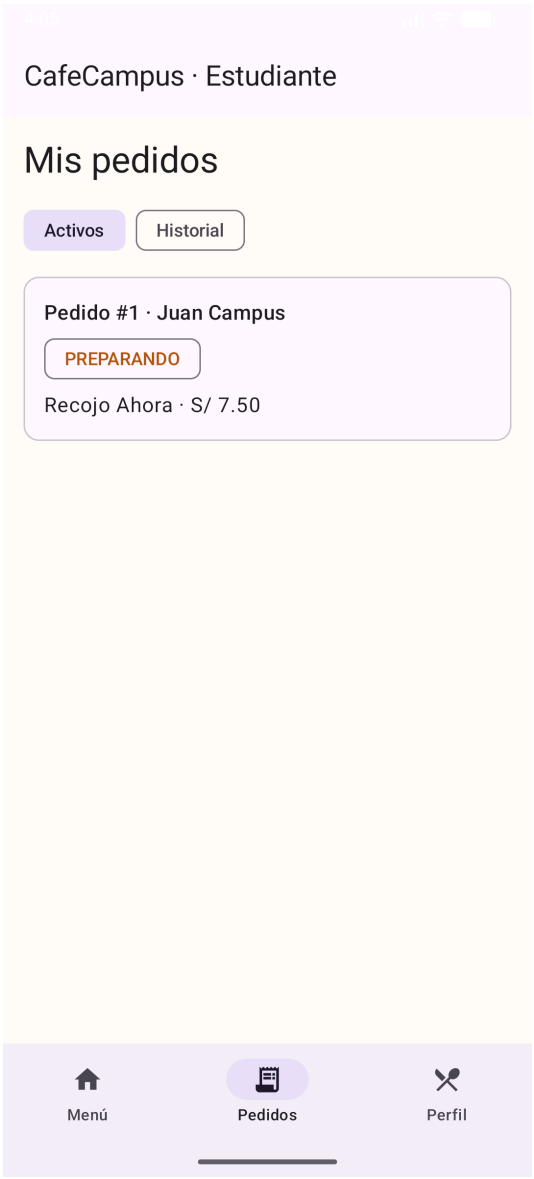
Administración del menú y de los pedidos

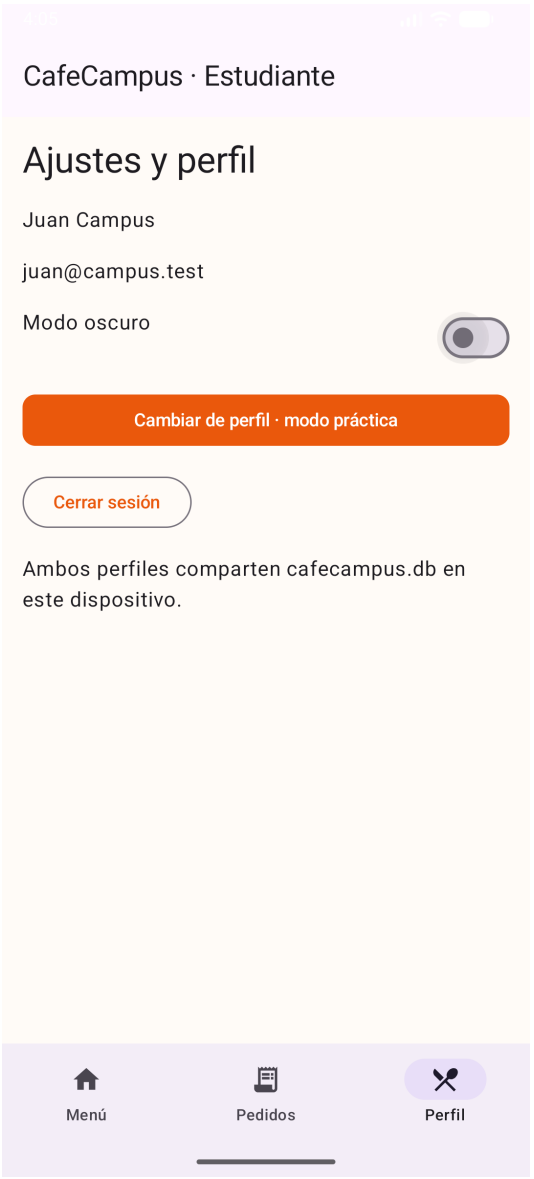




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

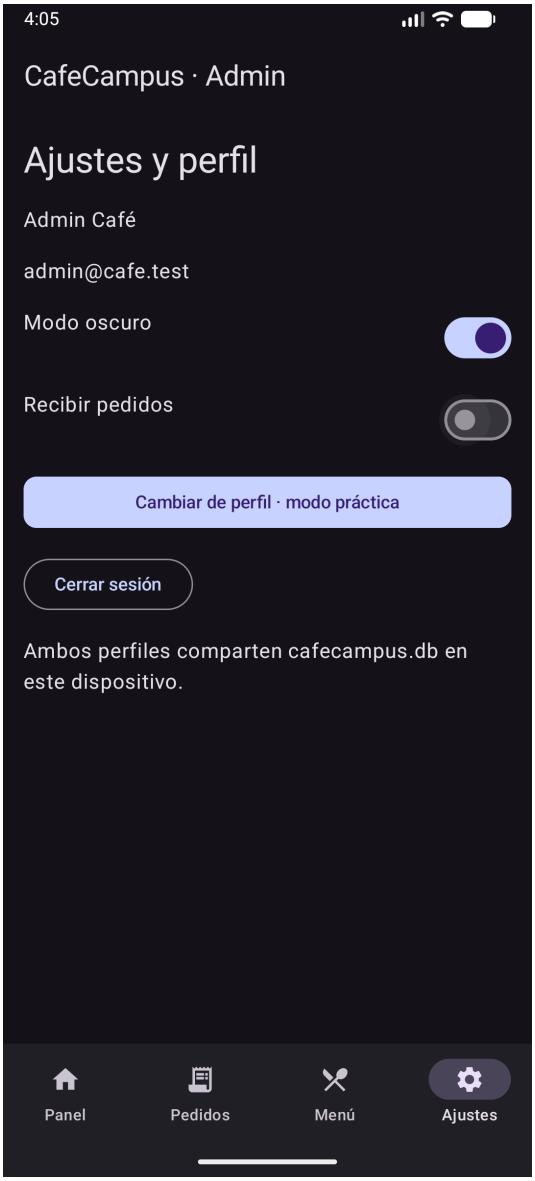
Cambio administrativo observado por el estudiante

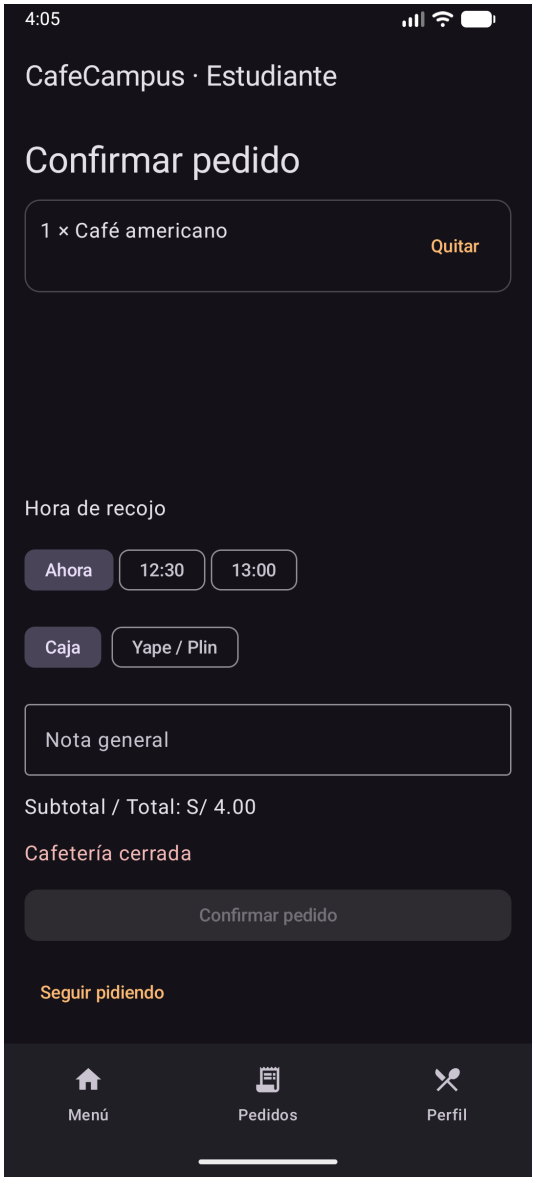




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

Recepción cerrada y confirmación bloqueada

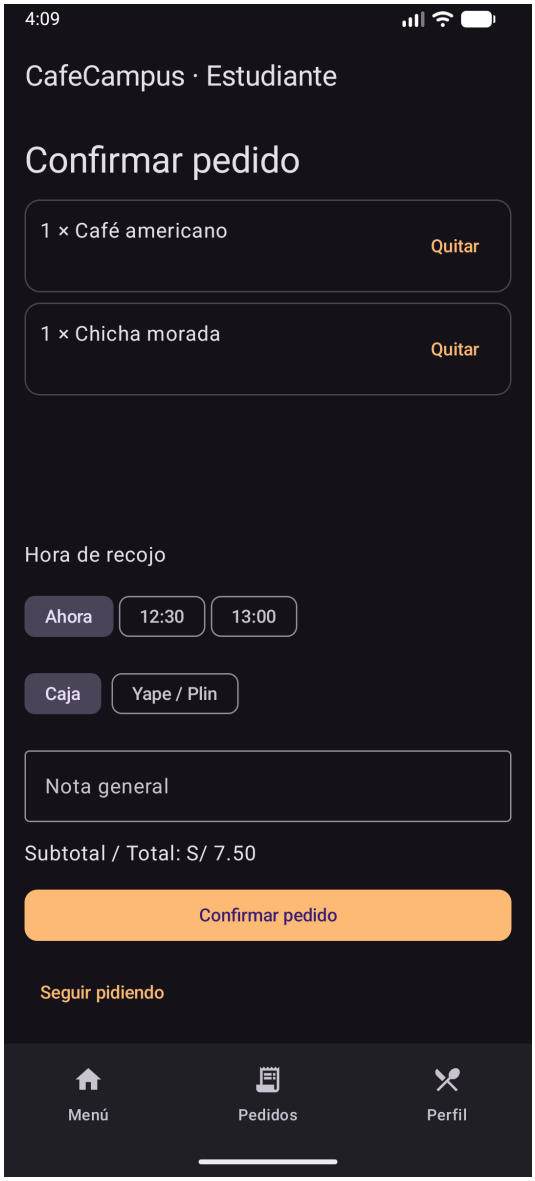




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

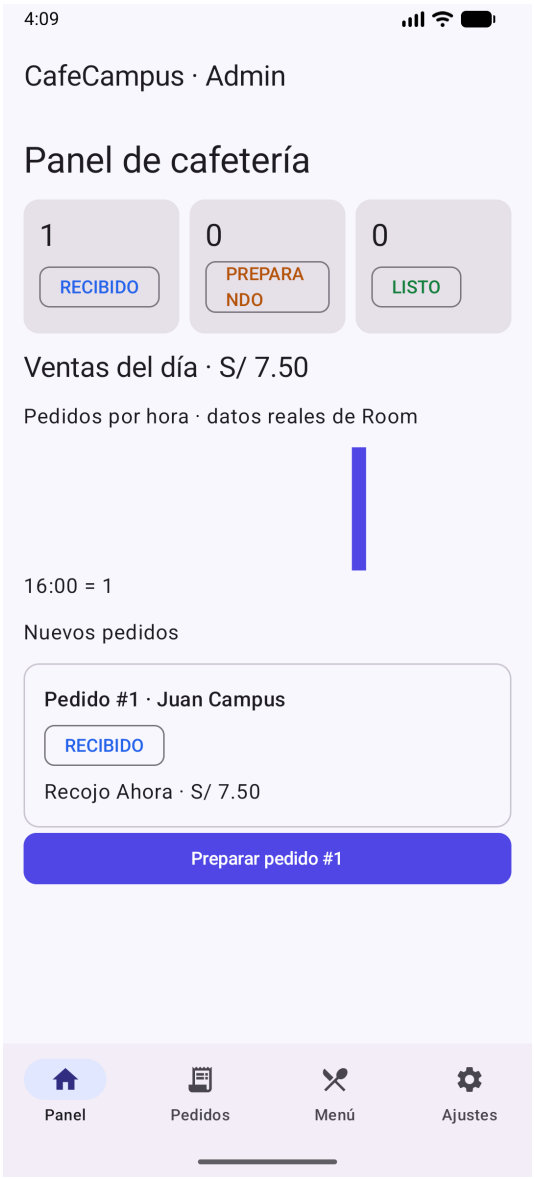
Mejoras búsqueda y repetición de pedidos

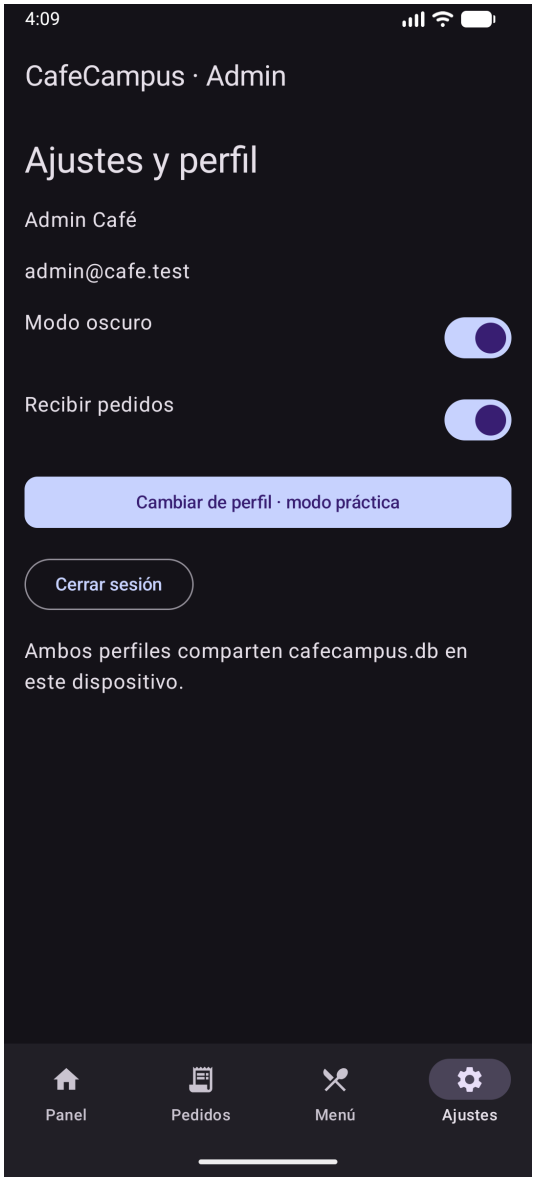




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

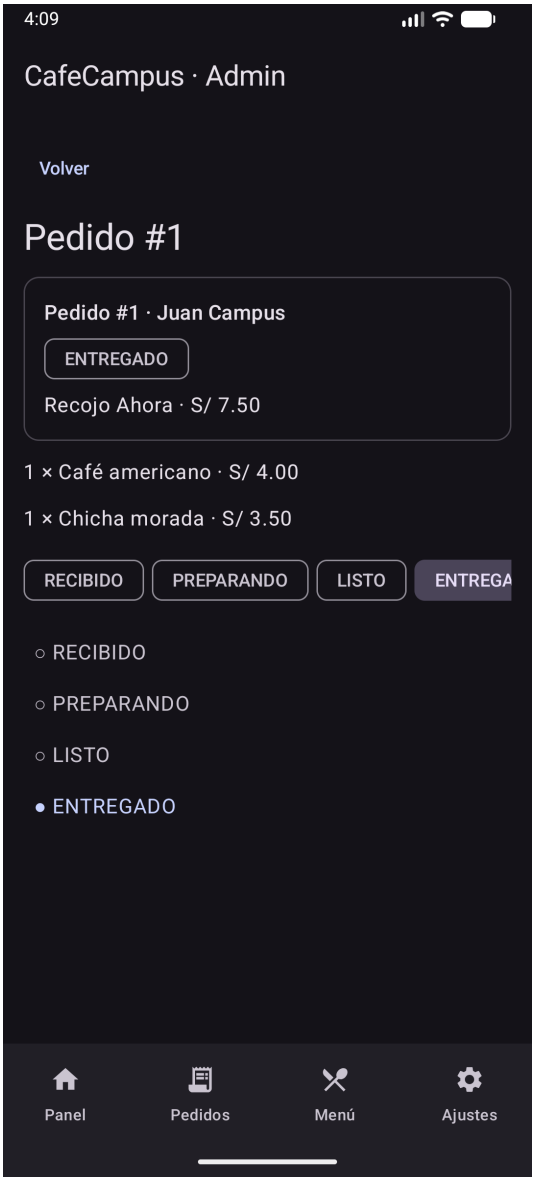
Mejoras panel con datos y modo oscuro

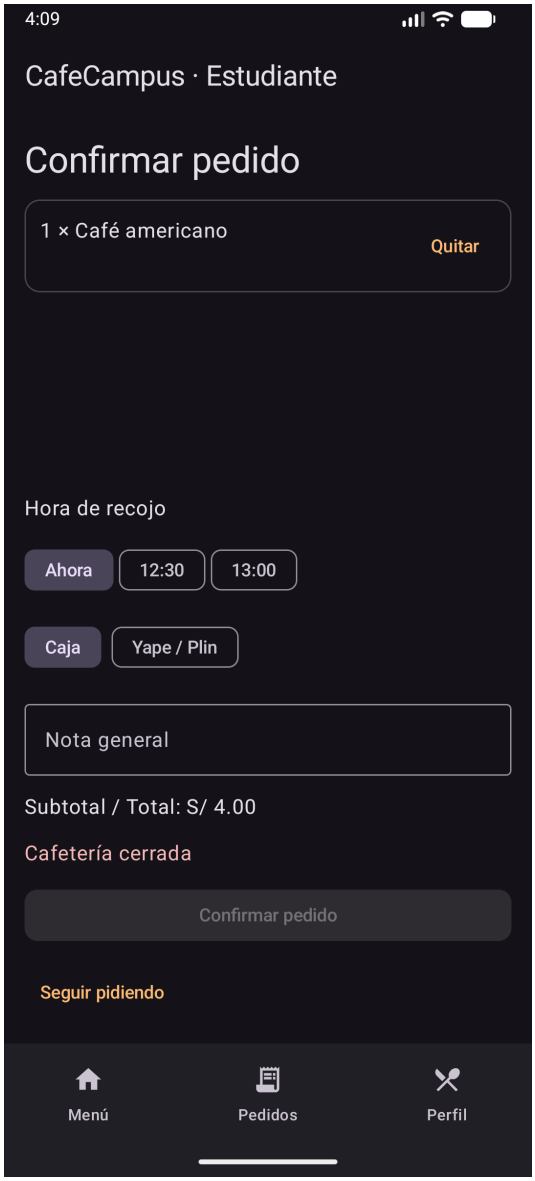




Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

Mejoras estado final y cafetería cerrada





Capturas reales del emulador Android obtenidas durante las pruebas instrumentadas del 23 de octubre de 2026.

Código PedidoDao.kt fragmento 1

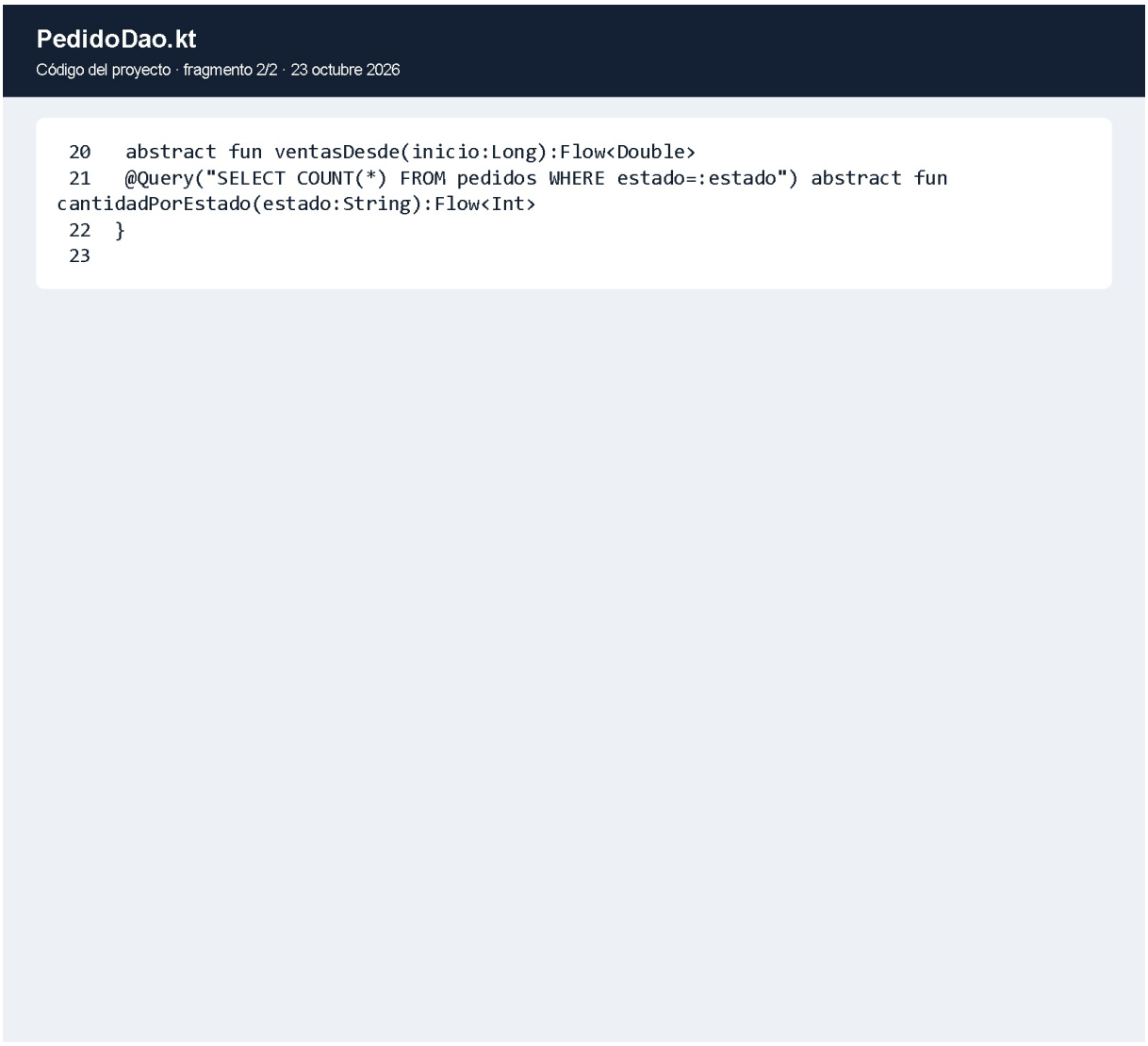
PedidoDao.kt

Código del proyecto · fragmento 1/2 · 23 octubre 2026

```
 1  package com.tecsup.cafecampus.data.local.dao
 2  import androidx.room.*
 3  import kotlinx.coroutines.flow.Flow
 4  import com.tecsup.cafecampus.data.local.entity.*
 5  import com.tecsup.cafecampus.data.local.relation.PedidoConDetalle
 6  @Dao abstract class PedidoDao {
 7   @Insert abstract suspend fun insertarPedido(p:Pedido):Long
 8   @Insert abstract suspend fun insertarDetalles(d:List<DetallePedido>)
 9   @Query("UPDATE pedidos SET estado=:estado WHERE id=:id") abstract suspend fun
cambiarEstado(id:Int,estado:String)
10   @Transaction open suspend fun crearPedido(p:Pedido,d:List<DetallePedido>):Long {
11    val id=insertarPedido(p); insertarDetalles(d.map { it.copy(pedidoId=id.toInt()) });
return id
12   }
13   @Transaction @Query("SELECT * FROM pedidos WHERE clienteId=:clienteId ORDER BY fecha
DESC")
14   abstract fun observarPorCliente(clienteId:Int):Flow<List<PedidoConDetalle>>
15   @Transaction @Query("SELECT * FROM pedidos WHERE estado=:estado ORDER BY fecha
DESC")
16   abstract fun observarPorEstado(estado:String):Flow<List<PedidoConDetalle>>
17   @Transaction @Query("SELECT * FROM pedidos WHERE id=:id")
18   abstract fun observarUno(id:Int):Flow<PedidoConDetalle?>
19   @Query("SELECT COALESCE(SUM(total),0.0) FROM pedidos WHERE fecha>=:inicio AND
estado!='CANCELADO'")
```

Captura del código del proyecto verificado, dentro de la guía original.

Código PedidoDao.kt fragmento 2



Captura del código del proyecto verificado, dentro de la guía original.

Código CafeRepositoryRoom.kt fragmento 1

CafeRepositoryRoom.kt

Código del proyecto · fragmento 1/2 · 23 octubre 2026

```
 1  package com.tecsup.cafecampus.data.repository
 2  import androidx.room.withTransaction
 3  import com.tecsup.cafecampus.data.local.CafeDatabase
 4  import com.tecsup.cafecampus.data.local.entity.*
 5  import com.tecsup.cafecampus.model.*
 6  class CafeRepositoryRoom(private val db:CafeDatabase):CafeRepository {
 7   override suspend fun
login(correo:String,clave:String)=db.usuarioDao().login(correo.trim(),clave)
 8   override suspend fun usuarioPorRol(rol:Rol)=db.usuarioDao().porRol(rol.name)
 9   override fun observarProductos()=db.productoDao().observarTodos()
10   override suspend fun guardarProducto(p:Producto) {
require(p.nombre.isNotBlank()&&p.precio.isFinite()&&p.precio>0) { "Nombre y precio válido
obligatorios" }; if(p.id==0) db.productoDao().insertar(p) else
db.productoDao().actualizar(p) }
11   override suspend fun eliminarProducto(p:Producto)=db.productoDao().eliminar(p)
12   override suspend fun
cambiarDisponibilidad(id:Int,disponible:Boolean)=db.productoDao().cambiarDisponibilidad(id
,disponible)
13   override suspend fun
crearPedido(clienteId:Int,items:List<ItemCarrito>,horaRecojo:String,pago:String,nota:Strin
g):Long = db.withTransaction {
14    require(items.isNotEmpty()&&items.all { it.cantidad>0 }) { "El pedido está vacío o
tiene cantidades inválidas" }
15    require(horaRecojo.isNotBlank()&&pago in listOf("CAJA","YAPE")) { "Selecciona hora
y pago" }
16    val actuales=items.map { item ->
17     val p=db.productoDao().obtener(item.producto.id)
```

Captura del código del proyecto verificado, dentro de la guía original.

Código CafeRepositoryRoom.kt fragmento 2

CafeRepositoryRoom.kt

Código del proyecto · fragmento 2/2 · 23 octubre 2026

```
 18     require(p!=null&&p.disponible) { "Un producto está agotado o fue eliminado" }
 19     item.copy(producto=p!!)
 20    }
 21    val detalles=actuales.map {
DetallePedido(productoId=it.producto.id,nombreProducto=it.producto.nombre,cantidad=it.cant
idad,precioUnitario=it.producto.precio) }
 22    val notas=listOf(nota.trim())+actuales.filter { it.nota.isNotBlank() }.map {
"${it.producto.nombre}: ${it.nota}" }
 23
db.pedidoDao().crearPedido(Pedido(clienteId=clienteId,horaRecojo=horaRecojo,pago=pago,nota
=notas.filter { it.isNotBlank() }.joinToString(" · "),total=actuales.sumOf {
it.producto.precio*it.cantidad }),detalles)
 24   }
 25   override fun
observarPedidosCliente(clienteId:Int)=db.pedidoDao().observarPorCliente(clienteId)
 26   override fun
observarPedidosPorEstado(estado:EstadoPedido)=db.pedidoDao().observarPorEstado(estado.name
)
 27   override fun observarPedido(id:Int)=db.pedidoDao().observarUno(id)
 28   override suspend fun
cambiarEstado(pedidoId:Int,estado:EstadoPedido)=db.pedidoDao().cambiarEstado(pedidoId,esta
do.name)
 29   override fun observarVentasHoy(inicioDia:Long)=db.pedidoDao().ventasDesde(inicioDia)
 30   override fun
observarCantidadPorEstado(estado:EstadoPedido)=db.pedidoDao().cantidadPorEstado(estado.nam
e)
 31  }
 32
```

Captura del código del proyecto verificado, dentro de la guía original.

Código PreferenciasStore.kt fragmento 1

PreferenciasStore.kt

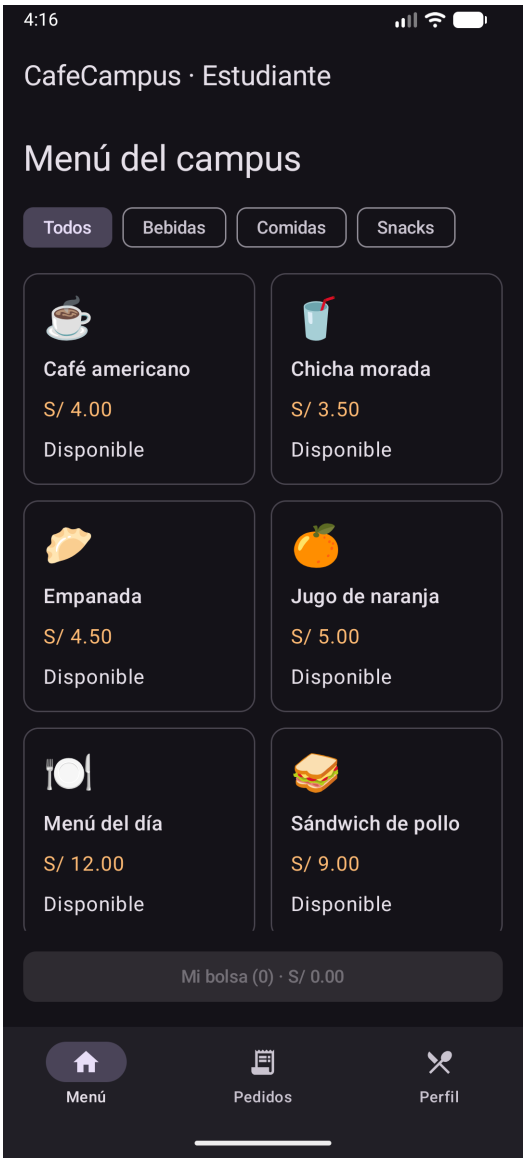
Código del proyecto · fragmento 1/1 · 23 octubre 2026

```
1  package com.tecsup.cafecampus.data.prefs
2  import android.content.Context
3  import androidx.datastore.preferences.core.*
4  import androidx.datastore.preferences.preferencesDataStore
5  import kotlinx.coroutines.flow.map
6  import com.tecsup.cafecampus.model.Rol
7  val Context.dataStore by preferencesDataStore("ajustes")
8  class PreferenciasStore(context:Context) {
9   private val store=context.applicationContext.dataStore
10  private val perfil=stringPreferencesKey("perfilActivo")
11  private val oscuro=booleanPreferencesKey("modoOscuro")
12  private val recibir=booleanPreferencesKey("recibirPedidos")
13  val perfilActivo=store.data.map { it[perfil]?:"" }
14  val modoOscuro=store.data.map { it[oscuro]?:false }
15  val recibirPedidos=store.data.map { it[recibir]?:true }
16  suspend fun guardarPerfil(rol:Rol)=store.edit { it[perfil]=rol.name }
17  suspend fun guardarModoOscuro(b:Boolean)=store.edit { it[oscuro]=b }
18  suspend fun guardarRecibirPedidos(b:Boolean)=store.edit { it[recibir]=b }
19  suspend fun cerrarSesion()=store.edit { it.remove(perfil) }
20 }
```

Captura del código del proyecto verificado, dentro de la guía original.

Persistencia comprobada tras cerrar el proceso

Se prepararon perfil CLIENTE, modo oscuro activado y recepción cerrada, se detuvo el proceso de CafeCampus y se inició una nueva ejecución. La prueba comprobó que DataStore conservó las tres preferencias y Room conservó el pedido confirmado. Resultado: OK, 1 prueba. Esto no afirma persistencia del carrito sin confirmar.



Captura real del nuevo proceso de CafeCampus. La prueba de reinicio se ejecuta después de PrepararPersistenciaTest y de detener el proceso, como se documenta en el proyecto.

Observaciones

1. El entorno utiliza datos ficticios y ambos roles comparten cafecampus.db. El cambio de perfil es una función del modo práctica y no un sistema de autorización para producción.

2. La persistencia de un pedido confirmado se logra con Room. El carrito se mantiene en ViewModel durante cambios de configuración, por lo que se distingue esa condición de la muerte del proceso.

3. La transacción evita insertar una cabecera sin sus líneas. Las pruebas rechazaron pedidos con productos agotados sin dejar registros parciales.

4. Se añadieron descripciones accesibles a los interruptores de modo oscuro y recepción para identificar sus acciones y probarlos de forma estable.

5. La fase inicial del repositorio ya fue asistida por ChatGPT, tal como declara su README. Esa autoría se mantiene visible; separar ramas no convierte el trabajo asistido en trabajo sin IA.

Conclusiones

1. La separación View, ViewModel, Repository y DAO permite cambiar la persistencia sin trasladar consultas SQL a las pantallas.

2. Flow mantiene sincronizados los perfiles locales: al pasar un pedido a Preparando, el estudiante observa el nuevo estado sin reiniciar la app.

3. Guardar nombre y precio unitario en el detalle conserva el valor histórico del pedido aunque cambie el catálogo.

4. Las comprobaciones del modo oscuro, recreación y cafetería cerrada convierten los requisitos de preferencias en comportamientos verificables.

5. Las mejoras de búsqueda, repetición y gráfico aportan utilidad y se mantuvieron dentro de MVVM. Los resultados de las pruebas respaldan los escenarios comprobados y no sustituyen la revisión personal del estudiante.

Registros de ejecución

Rama main

com.tecsup.cafecampus.FlujoUiTest:. com.tecsup.cafecampus.RoomAceptacionTest:.... Time: 64.686 OK (5 tests)

Rama mejora-ia

com.tecsup.cafecampus.FlujoUiTest:. com.tecsup.cafecampus.RoomAceptacionTest:..... Time: 41.477 OK (6 tests)

Fecha de verificación: 23 de octubre de 2026. Capturas, código y resultados corresponden al proyecto revisado. Los prompts y las mejoras se conservan en PROMPTS.md de la rama mejora-ia.